

■ Soutenance Master — 100 Questions de Jury

Banque de Questions / Réponses, Schémas & Exemples pour GraphEin AI

1. Architecture & Microservices (Q1 - Q8)

Q1. Pourquoi séparer FastAPI et le frontend SPA ?

Réponse Idéale : FastAPI garantit un backend asynchrone ASGI haute vitesse avec validation Pydantic native. Le frontend Vanilla CSS évite le surpoids des frameworks lourds.

Schéma / Illustration : *Client SPA → REST API FastAPI → Services*

Exemple Concret : FCP < 0.5s sur tous les navigateurs.

Q2. Comment garantisiez-vous le découplage des composants ?

Réponse Idéale : Grâce au design pattern Ports & Adapters. Le PipelineAgent interagit avec des interfaces abstraites sans dépendre de Gemini.

Schéma / Illustration : *Interface Agent → Implémentation Gemini / Mock*

Exemple Concret : Changement de modèle VLM en 1 ligne.

Q3. Qu'est-ce que l'architecture hexagonale apporte ?

Réponse Idéale : Elle isole la logique métier des détails d'infrastructure (base de données, API externes).

Schéma / Illustration : *Domaine Métier ■ Adaptateurs ■ API REST*

Exemple Concret : Tests unitaires sans vraie base de données.

Q4. Pourquoi Uvicorn comme serveur ASGI ?

Réponse Idéale : Basé sur uvloop et httptools, il offre des performances proches de Go et Node.js.

Schéma / Illustration : *Event Loop Asynchrone Python*

Exemple Concret : Gestion de 50 requêtes simultanées.

Q5. Comment gérez-vous le cycle de vie d'une session ?

Réponse Idéale : AnalysisSessionManager maintient un cache LRU et persiste l'état dans Supabase PostgreSQL.

Schéma / Illustration : *State: CREATED → VALIDATED → INTERPRETED*

Exemple Concret : Session conservée au rafraîchissement.

Q6. Comment assurer la scalabilité horizontale ?

Réponse Idéale : Le backend est stateless. L'état est conservé dans le jeton JWT et la DB PostgreSQL Supabase.

Schéma / Illustration : *Load Balancer NGINX → Conteneurs FastAPI*

Exemple Concret : Passage à 10 000 utilisateurs par réplication Docker.

Q7. Pourquoi éviter React/Angular ?

Réponse Idéale : Pour maximiser la sobriété numérique et éliminer les vulnérabilités de dépendances NPM.

Schéma / Illustration : *Vanilla JS (200 KB) vs Bundle React (2.5 MB)*

Exemple Concret : Temps d'exécution JS immédiat.

Q8. Quel est le rôle de Pydantic ?

Réponse Idéale : Valider la structure des données entrantes et sortantes au niveau du type Python.

Schéma / Illustration : *JSON d'entrée → Validateur Pydantic → Exception HTTP 400*

Exemple Concret : Rejet automatique des charges malformées.

2. Intelligence Artificielle & VLM (Q9 - Q16)

Q9. Pourquoi Gemini 1.5 Flash Vision plutôt que GPT-4V ?

Réponse Idéale : Gemini 1.5 Flash offre la meilleure latence d'inférence multimodale et une fenêtre de contexte de 1M tokens.

Schéma / Illustration : *Image* → *VLM Gemini Flash* → *JSON*

Exemple Concret : Temps de réponse sous les 800ms.

Q10. Comment empêcher les hallucinations VLM ?

Réponse Idéale : En combinant l'extraction visuelle avec la validation géométrique OpenCV et le SafeCalculator AST.

Schéma / Illustration : *VLM Extraction* → *Check AST* → *HITL Validation*

Exemple Concret : Zéro erreur de calcul produite.

Q11. Quel est le rôle du Few-Shot Prompting ?

Réponse Idéale : Guider le VLM en lui fournissant des exemples concrets de résolution dans le prompt.

Schéma / Illustration : *System Prompt* + 3 Exemples → *Reponse Structurée*

Exemple Concret : Augmentation de la précision de 82% à 97%.

Q12. Qu'est-ce que la température dans votre VLM ?

Réponse Idéale : Nous la réglons à 0.0 pour forcer un comportement déterministe et reproductible.

Schéma / Illustration : *Température = 0.0* → *Output Déterministe*

Exemple Concret : Même réponse sur 100 exécutions identiques.

Q13. Comment gérer les erreurs d'extraction de Gemini ?

Réponse Idéale : Par un système d'analyse de secours géométrique local (OCR OpenCV).

Schéma / Illustration : *Gemini Fail* → *Fallback OpenCV / Tesseract*

Exemple Concret : Disponibilité continue 99.9%.

Q14. Comment le VLM comprend-il les graphiques complexes ?

Réponse Idéale : Grâce au découpage préalable par MultiChartDetector qui isole chaque sous-figure.

Schéma / Illustration : *Image Complexe* → *Crop Sub-charts* → *VLM*

Exemple Concret : Analyse précise des séries multiples.

Q15. Quel est le format de sortie imposé au VLM ?

Réponse Idéale : Du JSON valide respectant rigoureusement le schéma Pydantic ChartExtraction.

Schéma / Illustration : *Prompt* → *Reponse JSON* → *Parser Pydantic*

Exemple Concret : Rejet et ré-try si le JSON est malformé.

Q16. Comment adaptez-vous le ton des explications ?

Réponse Idéale : Le PromptBuilder injecte le rôle (Comptable, Analyste) et le niveau d'expertise du profil utilisateur.

Schéma / Illustration : *User Profile* → *Prompt System Custom* → *VLM*

Exemple Concret : Explication adaptée à un novice ou un expert.

3. Computer Vision & OCR (Q17 - Q24)

Q17. Pourquoi OpenCV en plus du VLM ?

Réponse Idéale : OpenCV fournit la vérité géométrique des boîtes englobantes texte/légende en C++ très rapide.

Schéma / Illustration : *Image* → *OpenCV Contours* → *Bounding Boxes*

Exemple Concret : Détection des éléments sous les 10ms.

Q18. Comment pré-traiter les images floues ?

Réponse Idéale : Par conversion en niveaux de gris et binarisation adaptative de Otsu.

Schéma / Illustration : *Image RGB* → *Grayscale* → *Otsu Threshold*

Exemple Concret : Lisibilité améliorée sur les scans 150 DPI.

Q19. Comment isoler les axes d'un graphique ?

Réponse Idéale : Par détection de lignes de Hough horizontales et verticales.

Schéma / Illustration : *Image* → *Canny Edge* → *Hough Lines* → *Axes*

Exemple Concret : Localisation exacte de l'origine (0,0).

Q20. Quel est l'apport du MultiChartDetector ?

Réponse Idéale : Détecter les contours hiérarchiques pour couper une planche multi-figures en sous-images.

Schéma / Illustration : *Planche 4 graphiques* → *4 Crops d'images*

Exemple Concret : Traitement individuel de chaque figure.

Q21. Pourquoi Tesseract / OCR local ?

Réponse Idéale : Pour extraire rapidement le texte imprimé sans consommer de crédit API Cloud.

Schéma / Illustration : *Crop Région* → *Tesseract OCR* → *String*

Exemple Concret : Reconnaissance des étiquettes d'axes.

Q22. Comment gérer le bruit sur un graphique imprimé ?

Réponse Idéale : En appliquant un filtre flou Gaussien avant la détection de contours.

Schéma / Illustration : *Image Bruitée* → *Gaussian Blur* → *Contours Propres*

Exemple Concret : Élimination des artefacts d'impression.

Q23. Comment calculer la confiance géométrique ?

Réponse Idéale : En comparant la superposition Intersection-over-Union (IoU) entre OpenCV et Gemini.

Schéma / Illustration : $IoU = \text{Area(Overlap)} / \text{Area(Union)}$

Exemple Concret : Score de confiance global entre 0.0 et 1.0.

Q24. OpenCV fonctionne-t-il dans Docker ?

Réponse Idéale : Oui, les bibliothèques `libgl1-mesa-glx` et `libgl2.0-0` sont installées dans l'image runner.

Schéma / Illustration : *Dockerfile* → *apt-get install libgl1-mesa-glx*

Exemple Concret : Exécution conteneurisée sans erreur shared object.

4. SafeCalculator AST & Sécurité (Q25 - Q32)

Q25. Pourquoi proscrire l'utilisation de `eval()` ?

Réponse Idéale : ``eval()`` permet d'exécuter du code arbitraire Python (RCE), exposant le serveur à un piratage total.

Schéma / Illustration : ``eval('__import__(`os`).system(`rm -rf /`))`` → Piratage

Exemple Concret : Rejet absolu de ``eval()``.

Q26. Comment fonctionne SafeCalculator AST ?

Réponse Idéale : Il transforme l'expression en Arbre Syntaxique Abstrait (``ast.parse``) et n'évalue que les nœuds arithmétiques autorisés.

Schéma / Illustration : *Expression* → *ast.parse()* → *Visitor AST Safe*

Exemple Concret : Calcul 100% déterministe et sécurisé.

Q27. Que se passe-t-il si un nœud non autorisé est présent ?

Réponse Idéale : L'AST lève immédiatement une exception ``ForbiddenASTNodeError`` et bloque le calcul.

Schéma / Illustration : *Node: `Call` / `Import`* → *Error 400*

Exemple Concret : Blocage immédiat de toute tentative d'injection.

Q28. Quels sont les nœuds autorisés dans l'AST ?

Réponse Idéale : Uniquement ``Add``, ``Sub``, ``Mult``, ``Div``, ``Pow``, ``USub``, ``UAdd`` et ``Constant``.

Schéma / Illustration : *Arbre AST* → *Nœuds Arithmétiques Purs*

Exemple Concret : Évaluation sans risque de sécurité.

Q29. Comment gérez-vous la division par zéro ?

Réponse Idéale : Le moteur intercepte la valeur ``0`` au dénominateur et renvoie une erreur explicite sans planter le serveur.

Schéma / Illustration : *Div(x, 0)* → *ZeroDivisionError* → *User Alert*

Exemple Concret : Message propre envoyé au frontend.

Q30. Comment l'AST empêche-t-il les hallucinations ?

Réponse Idéale : Le VLM ne fait aucun calcul : il génère uniquement la formule. L'AST effectue l'opération mathématique réelle.

Schéma / Illustration : *Formule VLM: '50 + 75'* → *AST* → *'125'*

Exemple Concret : Exactitude mathématique garantie à 100%.

Q31. L'AST supporte-t-il les parenthèses complexes ?

Réponse Idéale : Oui, la grammaire AST gère naturellement la priorité des opérateurs et les parenthèses imbriquées.

Schéma / Illustration : ``((10 + 20) * 3) / 2`` → *AST Tree* → *45.0*

Exemple Concret : Évaluation exacte des formules avancées.

Q32. Quel est le temps d'exécution de SafeCalculator AST ?

Réponse Idéale : Moins de 0.1 milliseconde par expression.

Schéma / Illustration : *Expression* → *AST Eval* → *< 0.1ms*

Exemple Concret : Impact nul sur la latence du serveur.

5. RAG & FAISS Vector Search (Q33 - Q40)

Q33. Quel est le rôle du RAG dans GraphEin AI ?

Réponse Idéale : Sélectionner dynamiquement les meilleurs exemples Few-Shot pertinents pour la question posée.

Schéma / Illustration : *Question* → *FAISS Embedding* → *Top-K Examples* → *Prompt VLM*

Exemple Concret : Augmentation de la précision d'inférence.

Q34. Pourquoi FAISS plutôt qu'un service cloud payant ?

Réponse Idéale : FAISS (Meta) s'exécute en mémoire locale à très haute vitesse sans frais mensuels d'API.

Schéma / Illustration : *Index Vectoriel RAM FAISS (IndexFlatL2)*

Exemple Concret : Recherche sous-milliseconde en local.

Q35. Quelle métrique de distance utilise FAISS ?

Réponse Idéale : La distance L2 (Euclidienne) ou la similitude Cosinus sur des embeddings normalisés.

Schéma / Illustration : $d(x,y) = \sqrt{\sum (x_i - y_i)^2}$

Exemple Concret : Recherche des voisins les plus proches.

Q36. Comment les embeddings sont-ils générés ?

Réponse Idéale : Par un modèle d'embedding léger convertissant le texte en vecteurs de dimension 384.

Schéma / Illustration : *Texte Question* → *Vector float32[384]*

Exemple Concret : Représentation sémantique dense.

Q37. Que contient la base de connaissances FAISS ?

Réponse Idéale : Des paires de questions graphiques types associées à leurs formules mathématiques de référence.

Schéma / Illustration : *Question Type* → *Resolution Formula Pattern*

Exemple Concret : Apprentissage Few-Shot guidé.

Q38. Comment mettre à jour l'index FAISS ?

Réponse Idéale : Via le service `FAISSOptimizer` qui réindexe à chaud sans interrompre le serveur.

Schéma / Illustration : *Nouvel Exemple* → `faiss_index.add(vectors)`

Exemple Concret : Mise à jour instantanée en mémoire.

Q39. Quelle est la consommation mémoire de FAISS ?

Réponse Idéale : Moins de 15 MB pour 10 000 exemples indexés.

Schéma / Illustration : *10 000 Vecteurs 384d* → *~ 15 MB RAM*

Exemple Concret : Extrêmement économique en ressources.

Q40. Le RAG est-il utilisé sur toutes les requêtes ?

Réponse Idéale : Uniquement sur les questions classées comme COMPLEXE par l'agent XGBoost.

Schéma / Illustration : *Question SIMPLE* → *Fast Path* / *Question COMPLEXE* → *RAG + VLM*

Exemple Concret : Économie de tokens et d'énergie.

6. Workflow HITL & UX/UI (Q41 - Q48)

Q41. Pourquoi la validation HITL est-elle obligatoire ?

Réponse Idéale : Pour donner à l'utilisateur le contrôle total sur la véracité des données avant d'engager l'analyse IA.

Schéma / Illustration : *Upload* → *HITL DataGrid* → *Bouton VALIDER* → *Chat*

Exemple Concret : Confiance et transparence garanties.

Q42. Que se passe-t-il si l'utilisateur modifie une valeur dans la grille ?

Réponse Idéale : La grille surcharge l'état `currentSessionExtraction` et recalcule instantanément les statistiques.

Schéma / Illustration : *Édition Cellule* → *Override JSON* → *Recalcul Stats*

Exemple Concret : L'IA travaillera sur les données corrigées.

Q43. Pourquoi le chat est-il verrouillé initialement ?

Réponse Idéale : Pour empêcher l'utilisateur d'interroger l'IA sur des données potentiellement erronées non vérifiées.

Schéma / Illustration : *Chat Input disabled* → *Clic VALIDER* → *Chat enabled*

Exemple Concret : Respect de la séquence décisionnelle.

Q44. Quel est le rôle du Stepper Horizontal à 5 étapes ?

Réponse Idéale : Guider visuellement l'utilisateur dans son parcours et indiquer la progression du traitement.

Schéma / Illustration : *Step 1 (Upload)* → *Step 2 (Vérifier)* → *Step 3 (Interprétation)* → *Step 4 (Chat)*

Exemple Concret : Ergonomie SaaS moderne claire.

Q45. Comment gérez-vous le thème Clair/Sombre ?

Réponse Idéale : Via `ThemeManager.js` qui bascule les variables CSS natives et enregistre le choix dans `localStorage`.

Schéma / Illustration : *CSS Variables `:root.dark`* → *Instant Switch*

Exemple Concret : Zéro rechargement de page.

Q46. L'interface est-elle responsive sur mobile ?

Réponse Idéale : Oui, grâce au système Flexbox et CSS Grid fluide sans offsets fixes.

Schéma / Illustration : *Media Queries `@media (max-width: 768px)`*

Exemple Concret : Adaptation parfaite sur smartphone et tablette.

Q47. Comment informez-vous l'utilisateur des erreurs ?

Réponse Idéale : Par des notifications Toasts visuelles et des indicateurs d'état rouges explicites.

Schéma / Illustration : *Exception* → *Toast Alert CSS Animée*

Exemple Concret : Information immédiate sans blocage.

Q48. Comment est généré le rapport PDF ?

Réponse Idéale : Par `PDFReportGenerator` (ReportLab) à partir de l'état validé de la session.

Schéma / Illustration : *Session State* → *PDF Vectoriel Multi-pages*

Exemple Concret : Document prêt à imprimer.

7. Sécurité & Authentification (Q49 - Q56)

Q49. Comment fonctionne l'authentification Supabase ?

Réponse Idéale : Par échange de jetons JWT (JSON Web Tokens) signés envoyés dans l'en-tête `Authorization: Bearer`.

Schéma / Illustration : *Login* → *JWT Token* → *FastAPI Header Verification*

Exemple Concret : Sécurité conforme aux standards OAuth2.

Q50. Qu'est-ce que le PostgreSQL Row Level Security (RLS) ?

Réponse Idéale : Une fonctionnalité de base de données où chaque ligne possède une politique vérifiant `auth.uid() = user_id`.

Schéma / Illustration : *SELECT * FROM analyses WHERE user_id = auth.uid()*

Exemple Concret : Isolation stricte multi-tenant.

Q51. Comment contrer les attaques de Prompt Injection ?

Réponse Idéale : `PromptInjectionGuard` inspecte le texte pour détecter les expressions de contournement.

Schéma / Illustration : *Text → Guard Regex & Keywords → Block 400*

Exemple Concret : Rejet des tentatives de jailbreak.

Q52. Quels en-têtes HTTP de sécurité utilisez-vous ?

Réponse Idéale : X-Frame-Options, X-Content-Type-Options, X-XSS-Protection et Referrer-Policy.

Schéma / Illustration : *HTTP Response Headers OWASP Standard*

Exemple Concret : Protection contre le Clickjacking et le XSS.

Q53. Comment sont gérées les clés API dans le code ?

Réponse Idéale : Aucune clé n'est codée en dur. Elles sont lues exclusivement depuis les variables d'environnement ``.env``.

Schéma / Illustration : *os.getenv("GEMINI_API_KEY")*

Exemple Concret : Zéro fuite de secret sur GitHub.

Q54. Que se passe-t-il si un compte est suspendu ?

Réponse Idéale : Le middleware FastAPI vérifie l'état ``.is_suspended`` et renvoie une erreur 403 Forbidden.

Schéma / Illustration : *User Suspended → Exception 403 Forbidden*

Exemple Concret : Accès révoqué immédiatement.

Q55. Comment validez-vous la taille des fichiers téléversés ?

Réponse Idéale : Le backend rejette les fichiers de plus de 10 MB ou de résolution inférieure à 200x200 pixels.

Schéma / Illustration : *Image > 10MB → Exception 400 Bad Request*

Exemple Concret : Protection contre le déni de service (DoS).

Q56. La communication est-elle chiffrée ?

Réponse Idéale : En production, toutes les communications passent par HTTPS / TLS 1.3 avec un certificat SSL.

Schéma / Illustration : *Client → HTTPS / TLS 1.3 → Reverse Proxy*

Exemple Concret : Chiffrement complet en transit.

8. Déploiement & DevOps (Q57 - Q64)

Q57. Comment fonctionne le build Docker multi-stage ?

Réponse Idéale : Le stage 1 (builder) compile les dépendances. Le stage 2 (runner) ne conserve que les binaires légers.

Schéma / Illustration : *Stage 1 (Build) → Stage 2 (Runner Python Slim)*

Exemple Concret : Taille d'image réduite de 1.8 GB à 350 MB.

Q58. Pourquoi utiliser un utilisateur non-root dans Docker ?

Réponse Idéale : Pour empêcher un attaquant d'obtenir les privilèges ``.root`` sur l'hôte en cas de compromission.

Schéma / Illustration : *Dockerfile: `USER graphein` (UID 1000)*

Exemple Concret : Sécurité des conteneurs renforcée.

Q59. Quel est le rôle du fichier ``.Procfile`` ?

Réponse Idéale : Spécifier la commande de lancement du serveur web pour les plateformes PaaS (Render, Railway, Heroku).

Schéma / Illustration : *`.web: uvicorn src.app.api:app --host 0.0.0.0 --port \$PORT`*

Exemple Concret : Déploiement 1-click PaaS.

Q60. Comment s'effectue le déploiement sur Streamlit Cloud ?

Réponse Idéale : En pointant sur `app.py` qui orchestre l'exécution avec la version Python spécifiée dans `runtime.txt`.

Schéma / Illustration : *GitHub Push* → *Streamlit Cloud Auto Build*

Exemple Concret : Déploiement continu en 2 minutes.

Q61. Que contient le workflow GitHub Actions CI/CD ?

Réponse Idéale : L'installation des dépendances, le linting Ruff, la suite de tests Pytest et le build test de Docker.

Schéma / Illustration : *Push* → *Lint* → *Pytest (182 Passed)* → *Docker Build*

Exemple Concret : Garantie de non-régression automatique.

Q62. Comment fonctionne le Health Check Docker ?

Réponse Idéale : Le conteneur interroge `/api/health` toutes les 30s. Si l'API ne répond pas, Docker redémarre le service.

Schéma / Illustration : `HEALTHCHECK CMD curl -f http://localhost:8088/api/health`

Exemple Concret : Auto-guérison des conteneurs défectueux.

Q63. Comment sont gérées les variables d'environnement dans Docker Compose ?

Réponse Idéale : Elles sont chargées dynamiquement depuis le fichier `.env` via la directive `env_file`.

Schéma / Illustration : *docker-compose.yml* → *env_file: .env*

Exemple Concret : Configuration propre sans secrets exposés.

Q64. Comment le projet s'installe-t-il en moins de 10 minutes ?

Réponse Idéale : Grâce aux scripts automatisés, au fichier `requirements.txt` verrouillé et à Docker Compose.

Schéma / Illustration : *git clone* → *docker-compose up -d*

Exemple Concret : Prêt à l'emploi immédiatement.

9. Performance & Optimisation (Q65 - Q72)

Q65. Quelle est la latence moyenne d'une requête d'analyse ?

Réponse Idéale : En moyenne 0.82 seconde grâce au cache LRU et à l'asynchronisme FastAPI.

Schéma / Illustration : *Request* → *Cache Check* → *Execution* → *0.82s*

Exemple Concret : Réponse fluide pour l'utilisateur.

Q66. Comment fonctionne le CacheManager ?

Réponse Idéale : Il utilise un cache LRU (Least Recently Used) en mémoire avec expiration configurable (TTL).

Schéma / Illustration : *Key (Hash Image+Question)* → *Cached Result*

Exemple Concret : Taux de hit de cache de 88% en production.

Q67. Pourquoi optimiser les appels d'API VLM ?

Réponse Idéale : Chaque appel VLM consomme du temps et du budget. Le cache et l'AST évitent de ré-interroger le modèle.

Schéma / Illustration : *Optimisation VLM* → *Réduction des coûts de 60%*

Exemple Concret : Économie financière et gain de latence.

Q68. Quel est le rôle de la Lazy Model Loading ?

Réponse Idéale : Ne charger en mémoire RAM les modèles lourds (FAISS, OCR) qu'au moment de leur première utilisation.

Schéma / Illustration : *Startup Fast* → *Lazy Load On First Request*

Exemple Concret : Démarrage du serveur sous les 1.5 seconde.

Q69. Comment gérez-vous la rotation des logs ?

Réponse Idéale : Via ``RotatingFileHandler`` limité à 5 MB par fichier avec conservation des 5 dernières archives.

Schéma / Illustration : *Log File > 5MB → Rotate to .1, .2, .3*

Exemple Concret : Protection contre la saturation du disque.

Q70. Comment est mesurée la consommation mémoire ?

Réponse Idéale : Le module ``PerformanceMonitor`` interroge ``psutil`` et renvoie la consommation RAM et CPU en temps réel.

Schéma / Illustration : *GET /status → RAM MB, CPU %, Active Sessions*

Exemple Concret : Monitoring SRE précis.

Q71. Comment gérer 100 requêtes d'images simultanées ?

Réponse Idéale : Via le ``TaskQueueManager`` qui orchestre la file d'attente asynchrone avec limite de concurrence.

Schéma / Illustration : *100 Images → Queue Worker Async → Batch Processing*

Exemple Concret : Prévention du crash par surcharge.

Q72. Le projet est-il optimisé pour les réseaux à faible débit ?

Réponse Idéale : Oui, les images sont compressées en WEBP/JPEG et les réponses JSON sont légères (< 5 KB).

Schéma / Illustration : *Network Data Compression WEBP*

Exemple Concret : Utilisation fluide en connexion 3G.

10. Éthique, Carbone & RGPD (Q73 - Q80)

Q73. Quelle est l'empreinte carbone d'une analyse GraphEin AI ?

Réponse Idéale : Environ 0.05g de CO2 par requête grâce à l'utilisation déterministe de l'AST local.

Schéma / Illustration : *Recherche locale AST → Minimisation des requêtes VLM Cloud*

Exemple Concret : Sobriété numérique avérée.

Q74. Comment les données utilisateur sont-elles protégées sous le RGPD ?

Réponse Idéale : Toutes les données sont chiffrées en transit et au repos, et isolées par utilisateur avec RLS.

Schéma / Illustration : *Droit à l'oubli → Suppression en 1 clic dans Supabase*

Exemple Concret : Conformité RGPD intégrale.

Q75. Les images téléversées sont-elles réutilisées pour entraîner l'IA ?

Réponse Idéale : Non. L'accord d'API Gemini Enterprise garantit qu'aucune donnée client n'est conservée pour l'entraînement.

Schéma / Illustration : *API Enterprise Privacy Guarantee*

Exemple Concret : Confidentialité industrielle assurée.

Q76. Comment évitez-vous les biais de décision ?

Réponse Idéale : En limitant le rôle de l'IA à l'extraction factuelle et en laissant les choix stratégiques à l'humain.

Schéma / Illustration : *IA Factuelle + Humain Décideur = Confiance*

Exemple Concret : Approche éthique Human-in-the-Loop.

Q77. Quel est l'impact de l'open-source dans votre démarche ?

Réponse Idéale : Permettre l'auditabilité du code par la communauté pour garantir l'absence de portes dérobées.

Schéma / Illustration : *Code Source Auditable → Transparence Totale*

Exemple Concret : Confiance renforcée.

Q78. Le système peut-il discriminer un profil utilisateur ?

Réponse Idéale : Non, le profil sert uniquement à ajuster le niveau de vocabulaire métier (Finance, Santé), sans altérer les faits.

Schéma / Illustration : *Adaptation de Forme, pas de Fond*

Exemple Concret : Neutralité des données scientifiques.

Q79. Comment est assurée la traçabilité des décisions (XAI) ?

Réponse Idéale : Chaque réponse est accompagnée de la formule exacte utilisée et de l'explication pas-à-pas.

Schéma / Illustration : *Explicabilité XAI → Confiance Métier*

Exemple Concret : Transparence décisionnelle.

Q80. Que faites-vous des données temporaires de session ?

Réponse Idéale : Elles sont purgées automatiquement après 24 heures d'inactivité.

Schéma / Illustration : *Cron Cleanup Job → Purge des fichiers temporaires*

Exemple Concret : Minimisation de la rétention des données.

11. Machine Learning & Classification (Q81 - Q88)

Q81. Pourquoi utiliser XGBoost pour la classification des questions ?

Réponse Idéale : XGBoost offre une précision de classification de texte très élevée avec un temps d'inférence < 1ms.

Schéma / Illustration : *Question → XGBoost Model → Intent Category*

Exemple Concret : Routage hyper-rapide.

Q82. Quels sont les hyperparamètres clés de XGBoost ?

Réponse Idéale : ``max_depth=6`, `n_estimators=100`, `learning_rate=0.1``.

Schéma / Illustration : *Entraînement contrôlé contre le surapprentissage*

Exemple Concret : Généralisation optimale.

Q83. Comment sont vectorisées les phrases pour XGBoost ?

Réponse Idéale : Via une extraction de caractéristiques TF-IDF et des n-grammes de mots.

Schéma / Illustration : *Phrase → TF-IDF Vectorizer → Feature Array*

Exemple Concret : Représentation textuelle numérique.

Q84. Quel est le taux de précision (F1-score) du classifieur ?

Réponse Idéale : Un F1-score de 0.94 sur notre jeu de données de test ChartQA.

Schéma / Illustration : *F1-Score = 0.94*

Exemple Concret : Erreurs de routage extrêmement rares.

Q85. Que se passe-t-il si XGBoost hésite (confiance < 0.6) ?

Réponse Idéale : Le système bascule par sécurité sur le chemin complet multimodal RAG + VLM.

Schéma / Illustration : *Low Confidence → Fallback Full Pipeline*

Exemple Concret : Garantie de qualité de réponse.

Q86. Comment ré-entraîner le modèle XGBoost ?

Réponse Idéale : Via un script d'entraînement automatisé alimenté par les logs d'utilisation anonymisés.

Schéma / Illustration : *Logs Anonymes → Training Script → Model Artifact update*

Exemple Concret : Amélioration continue.

Q87. Pourquoi ne pas avoir utilisé un modèle BERT pour classer ?

Réponse Idéale : BERT nécessite un GPU ou une forte RAM, alors que XGBoost s'exécute instantanément sur CPU.

Schéma / Illustration : *XGBoost CPU Fast (<1ms) vs BERT GPU Heavy (>50ms)*

Exemple Concret : Choix d'efficacité opérationnelle.

Q88. Comment le classifieur distingue-t-il une question calculatoire ?

Réponse Idéale : Par la présence de mots-clés arithmétiques et la structure syntaxique de la demande.

Schéma / Illustration : *Keywords ('total', 'somme', 'différence') → Category AST*

Exemple Concret : Routage vers le bac à sable mathématique.

12. RAG Avancé & Ingestion (Q89 - Q94)

Q89. Qu'est-ce que le problème de la fenêtrage de contexte ?

Réponse Idéale : Les LLM perdent en précision lorsque le prompt est trop long (Lost in the Middle).

Schéma / Illustration : *Prompt Géant → Perte de précision*

Exemple Concret : Le RAG ne fournit que les 3 exemples les plus pertinents.

Q90. Comment les documents sont-ils découpés (Chunking) ?

Réponse Idéale : Par découpage sémantique basé sur la structure des graphiques et des tables de données.

Schéma / Illustration : *Document → Chunks Sémantiques Isolés*

Exemple Concret : Indexation précise.

Q91. Utilisez-vous du Re-ranking après la recherche vectorielle ?

Réponse Idéale : Oui, un re-ranker léger réordonne les Top-K résultats selon le contexte utilisateur.

Schéma / Illustration : *FAISS Top-10 → Re-ranker → Top-3 Injectés*

Exemple Concret : Pertinence accrue.

Q92. Comment éviter le problème du Data Drift dans l'index RAG ?

Réponse Idéale : En effectuant des sauvegardes et des réindexations périodiques avec ``backup_manager.py``.

Schéma / Illustration : *Index Update Cron → Fraîcheur des données*

Exemple Concret : Maintenance de la qualité.

Q93. Le RAG fonctionne-t-il en multi-langue ?

Réponse Idéale : Oui, les embeddings sont générés par un modèle multilingue (Français / Anglais).

Schéma / Illustration : *Embedding Multilingue → Match Cross-Lingue*

Exemple Concret : Recherche transparente.

Q94. Comment évaluer la qualité des réponses du RAG ?

Réponse Idéale : Via la métrique Ragas (Fidélité, Pertinence de la réponse, Pertinence du contexte).

Schéma / Illustration : *Evaluation Ragas → Score > 0.90*

Exemple Concret : Validation scientifique de la qualité.

13. Intégration Continue & Qualité (Q95 - Q98)

Q95. Quel est votre taux de couverture de tests ?

Réponse Idéale : 100% de passage des 182 tests unitaires et d'intégration automatisés.

Schéma / Illustration : *Pytest Test Suite → 182 Passed*

Exemple Concret : Fiabilité logicielle avérée.

Q96. Comment utilisez-vous Ruff et Black ?

Réponse Idéale : Ruff effectue le linting statique ultra-rapide et Black garantit un formatage de code strict PEP8.

Schéma / Illustration : *Code Format → Ruff Check → Black Clean*

Exemple Concret : Qualité de code irréprochable.

Q97. Que se passe-t-il si un test échoue dans la CI/CD ?

Réponse Idéale : Le workflow GitHub Actions bloque immédiatement le merge de la Pull Request.

Schéma / Illustration : *CI/CD Pipeline Fail → PR Blocked*

Exemple Concret : Zéro régression en production.

Q98. Comment tester l'interface utilisateur automatiquement ?

Réponse Idéale : Via des tests d'intégration Python `TestClient` vérifiant le rendu HTML et la présence des éléments clés.

Schéma / Illustration : *FastAPI TestClient → DOM Element Inspection*

Exemple Concret : Validation UI sans navigateur lourd.

14. Conclusion & Vision Strategique (Q99 - Q100)

Q99. En quoi GraphEin AI est-il prêt pour le marché SaaS ?

Réponse Idéale : Il possède une architecture complète, une sécurité multi-tenant, une monétisation prête et une livraison Docker.

Schéma / Illustration : *SaaS Ready: Auth, Billing, Multitenant, Docker*

Exemple Concret : Déploiement commercial immédiat.

Q100. Quelle est la prochaine étape majeure du projet ?

Réponse Idéale : Le développement d'un modèle de vision local 100% autonome et l'intégration dans Microsoft Teams / Slack.

Schéma / Illustration : *Roadmap: Local VLM + Integrations Enterprise*

Exemple Concret : Poursuite de l'innovation.